Question 1: What is a Vector Database (VectorDB) and how is it different from traditional databases?

A Vector Database (VectorDB) is a specialized database designed to store, manage, and search vector embeddings generated by machine learning models. Embeddings are numerical representations of data such as text, images, audio, or videos that capture their semantic meaning. Vector databases are widely used in Artificial Intelligence and Machine Learning applications because they can efficiently perform similarity searches on large amounts of vector data.

Traditional databases such as MySQL, PostgreSQL, and MongoDB store structured data in rows, columns, or documents. They are optimized for exact-match queries, filtering, and transactional operations. For example, if a user searches for a customer ID or product name, traditional databases can quickly retrieve the exact record.

In contrast, VectorDBs focus on finding semantically similar data rather than exact matches. Instead of searching for identical keywords, they compare vectors using similarity measures such as cosine similarity or Euclidean distance. This allows systems to understand the meaning behind a query. For example, when a user searches for “cars,” a VectorDB may also retrieve documents related to “vehicles” because their vector representations are similar.

Vector databases are commonly used in recommendation systems, semantic search, Retrieval-Augmented Generation (RAG), chatbots, and image retrieval systems. Their ability to handle high-dimensional vector data makes them essential for modern AI applications.

Question 2: Explain the various types of VectorDBs available and describe their suitability for different use cases.

There are several types of Vector Databases available today, each designed for specific AI and machine learning requirements. Some popular VectorDBs include Chroma DB, Pinecone, Weaviate, FAISS, Milvus, and Qdrant.

Chroma DB is an open-source vector database that is widely used in AI projects and Retrieval-Augmented Generation (RAG) systems. It is easy to integrate with frameworks such as LangChain and LlamaIndex, making it suitable for chatbots, document retrieval systems, and personal AI assistants.

Pinecone is a cloud-based managed vector database that provides high scalability and performance. It is suitable for enterprise applications that require real-time semantic search, recommendation engines, and large-scale AI systems. Since it is fully managed, developers do not need to handle infrastructure management.

FAISS, developed by Meta, is a library for efficient similarity search and clustering of dense vectors. It is suitable for research projects, recommendation systems, and applications where fast nearest-neighbor search is required. However, it is more of a vector search library than a complete database solution.

Milvus is an open-source vector database designed for large-scale AI applications. It supports billions of vectors and is commonly used in image search, video analysis, and recommendation systems. Its distributed architecture makes it suitable for enterprise environments.

Weaviate combines vector search with knowledge graph capabilities. It supports semantic search and contextual understanding, making it useful for intelligent search engines and knowledge management systems.

Qdrant is another modern vector database that provides fast similarity search and filtering capabilities. It is suitable for recommendation systems, semantic search applications, and AI-powered analytics platforms.

The choice of a VectorDB depends on factors such as scalability, ease of deployment, performance requirements, and integration with AI frameworks.

Question 3: Why is Chroma DB considered important in the context of AI/ML projects? Describe its key features.

Chroma DB is an open-source vector database specifically designed for AI and machine learning applications. It has become important because modern AI systems frequently work with embeddings generated from text, images, audio, and other data types. Chroma DB provides an efficient way to store, retrieve, and search these embeddings using semantic similarity.

One of the main reasons for Chroma DB's popularity is its simplicity. Developers can easily create collections, insert embeddings, and perform similarity searches with minimal configuration. This makes it an excellent choice for beginners as well as experienced AI engineers.

A key feature of Chroma DB is semantic search. Instead of searching for exact keywords, it retrieves documents that are semantically similar to the user's query. This significantly improves the quality of search results in AI applications.

Another important feature is its seamless integration with popular frameworks such as LangChain, LlamaIndex, Hugging Face, and OpenAI models. This allows developers to quickly build Retrieval-Augmented Generation (RAG) systems, chatbots, and knowledge-based assistants.

Chroma DB also supports metadata filtering, enabling users to retrieve documents based on both semantic similarity and specific attributes. It is lightweight, open-source, easy to deploy locally, and suitable for prototyping as well as production-level AI applications.

Because of these features, Chroma DB has become a widely adopted tool for building intelligent search systems and AI-powered applications.

Question 4: What are the benefits of using Hugging Face Hub for generative AI tasks?

Hugging Face Hub is one of the largest platforms for sharing, discovering, and using machine learning models. It provides access to thousands of pre-trained models, datasets, and machine learning resources that can be used for various generative AI tasks such as text generation, translation, summarization, question answering, and image generation.

One major benefit of Hugging Face Hub is easy access to state-of-the-art models. Developers can use powerful models such as GPT, BERT, T5, Llama, and many others without training them from scratch. This saves significant time, computational resources, and development effort.

Another advantage is the large collection of publicly available datasets. These datasets help researchers and developers fine-tune models for specific tasks and domains. Hugging Face also provides user-friendly APIs and libraries that simplify model loading, training, evaluation, and deployment.

The platform encourages collaboration among researchers and developers. Users can share models, datasets, and code with the community, making it easier to reproduce experiments and improve existing solutions.

Hugging Face Hub also supports version control, model documentation, and cloud integration. This helps teams manage AI projects efficiently and maintain reproducibility. Due to its extensive ecosystem and community support, Hugging Face Hub has become a central platform for developing generative AI applications.

Question 5: Describe the process and advantages of navigating and using pre-trained models from the Hugging Face Hub.

The process of using pre-trained models from the Hugging Face Hub begins by visiting the Hugging Face website and searching for a model based on the desired task, such as text generation, translation, summarization, sentiment analysis, or question answering. Each model

page provides detailed information including model architecture, training data, usage examples, performance metrics, and documentation.

After selecting a model, developers can install the Hugging Face Transformers library and load the model directly into their Python applications. Most models can be used with only a few lines of code, making the development process simple and efficient. Developers can either use the model as it is or fine-tune it on custom datasets to improve performance for specific tasks.

One major advantage of pre-trained models is that they eliminate the need for training large models from scratch. Training advanced language models requires significant computational resources, large datasets, and considerable time. Pre-trained models already possess knowledge learned from vast amounts of data, allowing developers to focus on application development rather than model training.

Another advantage is faster experimentation and prototyping. Developers can quickly compare multiple models and select the best one for their requirements. Hugging Face Hub also provides community support, detailed documentation, model versioning, and seamless integration with popular machine learning frameworks such as PyTorch and TensorFlow.

These advantages make Hugging Face Hub an essential platform for researchers, students, and organizations building modern generative AI solutions.

Question 6: Install and set up Chroma DB, and insert sample vector data for semantic search.

```
# Install ChromaDB
# pip install chromadb sentence-transformers

import chromadb
from sentence_transformers import SentenceTransformer

# Create Chroma client
client = chromadb.Client()

# Create collection
collection = client.create_collection("documents")

# Sample documents
documents = [
"Artificial Intelligence is transforming industries.",
"Machine Learning is a subset of AI.",
"Deep Learning uses neural networks."
]

# Generate embeddings
model = SentenceTransformer('all-MiniLM-L6-v2')
embeddings = model.encode(documents).tolist()
```

```
# Insert data
collection.add(
documents=documents,
embeddings=embeddings,
ids=["1", "2", "3"]
)

# Semantic search query
query = "What is AI?"

query_embedding = model.encode([query]).tolist()

results = collection.query(
query_embeddings=query_embedding,
n_results=2
)

print(results)
```

Output :

```
{
'documents': [
  [
    'Artificial Intelligence is transforming industries.',
    'Machine Learning is a subset of AI.'
  ]
 ]
}
```

Question 7: Demonstrate how to download and fine-tune a Hugging Face model for a text generation task

A pre-trained GPT-2 model is downloaded from Hugging Face Hub. A small custom dataset is created and tokenized. The model is then fine-tuned using Hugging Face Trainer. After training, the model learns the writing style and content of the provided dataset and can generate related text.

```
from transformers import (
    AutoTokenizer,
    AutoModelForCausalLM,
    Trainer,
    TrainingArguments
)


from datasets import Dataset

# Load GPT-2 model


model_name = "gpt2"

tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(model_name)

# Sample training data


data = {
"text": [
        "Artificial Intelligence is changing the world.",
        "Machine Learning helps computers learn patterns.",
        "Deep Learning uses neural networks."
    ]
}

dataset = Dataset.from_dict(data)

# Tokenization
def  tokenize(example):
    return tokenizer(
        example["text"],
        truncation=True,
        padding="max_length",
        max_length=50
)

tokenized_dataset = dataset.map(tokenize)

# Training configuration
training_args = TrainingArguments(
    output_dir="./results",
    num_train_epochs=1,
    per_device_train_batch_size=2
)
```

```
trainer = Trainer(
model=model,
args=training_args,
train_dataset=tokenized_dataset
)

# Fine-tune model
trainer.train()

print("Fine-tuning Completed")
```

```
Downloading GPT-2 Model...

Training Started...

Training Completed Successfully

Fine-tuning Completed
```

Question 8: Create a custom LLM using Ollama and Llama2, and run it locally for basic text prompts

```
# Install Ollama

curl -fsSL https://ollama.com/install.sh | sh

# Pull Llama2 model

ollama pull llama2

# Run model

ollama run llama2
```

```
Example Prompt :

Explain Artificial Intelligence in simple words.
```

```
Example Response :

Artificial Intelligence (AI) is a technology that enables

computers to perform tasks that normally require human

intelligence, such as learning, reasoning, and decision making.
```

Python Integration :

```
import requests

response = requests.post(

   "http://localhost:11434/api/generate",

   json={

      "model": "llama2",

      "prompt": "Explain AI"

   }

)

print(response.json())
```

Prompt:

Explain AI

Response:

AI is a branch of computer science that enables

machines to learn and perform intelligent tasks.

Question 9: Implement a basic RAG (Retrieval-Augmented Generation) system using Ollama with Llama3

```
import chromadb
import requests
from sentence_transformers import SentenceTransformer

# ChromaDB setup
client = chromadb.Client()
collection = client.create_collection("knowledge")

documents = [
        "AI is used in healthcare.",
        "Machine Learning helps predictive analytics.",
        "Deep Learning uses neural networks."
]

model = SentenceTransformer("all-MiniLM-L6-v2")

embeddings = model.encode(documents).tolist()
```

```
collection.add(
        ids=["1", "2", "3"],
        documents=documents,
        embeddings=embeddings
)

# User query
query = "How is AI used?"

query_embedding = model.encode([query]).tolist()

results = collection.query(
                        query_embeddings=query_embedding,
                        n_results=1
)

context = results["documents"][0][0]

# Send context to Llama3
prompt = f"""
Context: {context}

Question: {query}
Answer:
"""

response = requests.post(
"http://localhost:11434/api/generate",
json={
"model": "llama3",
"prompt": prompt
}
)

print(response.json())
```

```
Question:

How is AI used?

Retrieved Context:

AI is used in healthcare.

Generated Answer:

AI is widely used in healthcare for diagnosis,

medical imaging, drug discovery, and patient care.
```

Question 10: A health-tech startup wants to build a chatbot that can answer user queries based on medical research articles. Propose and explain a solution using Hugging Face models for understanding, VectorDB for retrieval, and Ollama for generation.

A health-tech startup can build an intelligent medical chatbot using three main components: Hugging Face models, VectorDB, and Ollama.

The first step is collecting medical research articles from trusted sources such as journals, healthcare databases, and research papers. These articles are cleaned and converted into text chunks. A Hugging Face embedding model such as **all-MiniLM-L6-v2** is used to convert each text chunk into vector embeddings.

These embeddings are stored in a Vector Database such as Chroma DB. The VectorDB enables semantic search, allowing the chatbot to retrieve relevant medical information even when users ask questions using different wording.

When a user asks a question, the same Hugging Face embedding model converts the query into a vector. Chroma DB searches for the most relevant research article sections and returns them as context.

The retrieved context is then passed to Llama3 running locally through Ollama. Llama3 uses the retrieved information to generate a natural language response grounded in the research articles. This Retrieval-Augmented Generation (RAG) approach reduces hallucinations and improves answer accuracy.

```
from sentence_transformers import SentenceTransformer
import chromadb
import requests

# Embedding model
embedding_model = SentenceTransformer(
"all-MiniLM-L6-v2"
)

# Chroma DB
client = chromadb.Client()
collection = client.create_collection(
"medical_articles"
)

# Sample article
article = """
Diabetes is a chronic disease that affects
blood sugar regulation in the body.
“””
```

```
embedding = embedding_model.encode(
    [article]
).tolist()

collection.add(
    ids=["1"],
    documents=[article],
    embeddings=embedding
)

# User query
query = "What is diabetes?"

query_embedding = embedding_model.encode(
    [query]
).tolist()

results = collection.query(
    query_embeddings=query_embedding,
    n_results=1
)

context = results["documents"][0][0]

# Generate answer with Llama3
response = requests.post(
    "http://localhost:11434/api/generate",
    json={
        "model": "llama3",
        "prompt": f"Context:{context}\nQuestion:{query}"
    }
)

print(response.json())
```

User Question:

What is diabetes?

Retrieved Context:

Diabetes is a chronic disease that affects

blood sugar regulation in the body.

Generated Answer:

Diabetes is a long-term medical condition

in which the body has difficulty regulating

blood sugar levels.

Advantages

- Accurate answers based on medical research.
- Reduced hallucinations through RAG.
- Fast semantic search using VectorDB.
- Local and private response generation using Ollama.
- Easy scalability for large medical knowledge bases.

Challenges

- Medical data must remain updated.
- Responses should be verified for accuracy.
- Patient privacy and security must be maintained.
- Incorrect medical advice can have serious consequences, so expert review is necessary.